

Aichestra

페쇄 기업형 LLM Provider 통합 설계서

API Key · OAuth/WIF/IAM · Local CLI Provider 통합 아키텍처

문서 버전	v1.0 Draft
작성일	2026-05-11 (Asia/Seoul)
대상 시스템	Aichestra Provider / Local Agent / Credential Manager
적용 범위	Codex, Claude, Gemini 및 기타 CLI 형 기업형 LLM Provider
핵심 원칙	Provider 호출과 Credential · Policy · Execution 을 분리

문서 요약

Aichestra 는 API key, OAuth/WIF/IAM, 로컬 CLI 를 모두 Provider 접근 방식으로 흡수할 수 있다. 단, 순수 웹앱은 사용자 터미널에 직접 접근할 수 없으므로 Aichestra Local Agent 가 필요하고, CLI 인증 토큰을 읽거나 복사하지 않는 "external CLI session" 방식이 기본 원칙이다.

목차

1. 핵심 결론
2. 범위와 전제
3. 통합 아키텍처
4. Provider 접근 방식 총괄
5. Provider 별 설계: Codex, Claude, Gemini, 기타 CLI
6. Local CLI Provider 상세 설계
7. Credential, 보안, 정책 모델
8. 설정 스키마와 예시
9. 구현 플로우와 코드 예시
10. 테스트, 운영, 단계별 로드맵
11. 리스크와 의사결정 표
12. 참고 출처

읽는 방법

API/OAuth 중심 설계만 검토하려면 3~5 장과 7 장을 보면 된다. 사용자 PC 에 설치된 Claude Code, Codex CLI, Gemini CLI 를 Aichestra 에서 실행·읽기하려면 6 장과 9 장을 중심으로 보면 된다.

1. 핵심 결론

Aichestra의 폐쇄 기업형 LLM Provider 설계는 API Key, OAuth/WIF/IAM, Local CLI Provider를 하나의 Provider 추상화로 묶되, 인증·권한·실행·정책을 분리해야 한다.

- **가능 여부:** 가능하다. 다만 Local CLI 방식은 Aichestra가 모델 API를 직접 호출하는 것이 아니라 사용자 PC의 인증된 CLI 프로세스를 실행하고 stdin/stdout/stderr를 중계하는 방식이다.
- **필수 조건:** Aichestra가 순수 웹앱이면 사용자 터미널에 직접 접근할 수 없다. Cloud/Web UI가 있다면 사용자 PC에서 실행되는 Aichestra Local Agent가 필요하다.
- **권장 우선순위:** 공식 API/SDK → 공식 OAuth/WIF/IAM → 공식 headless CLI → PTY 기반 interactive terminal automation 순으로 적용한다.
- **가장 중요한 보안 원칙:** Aichestra는 ~/.codex/auth.json, ~/.claude, Google credential cache 등 CLI 인증 저장소를 읽거나 서버로 업로드하지 않는다.
- **Gemini 주의사항:** Gemini CLI의 Google OAuth 세션을 제 3자 backend 접근 수단으로 harvest/piggyback하는 방식은 Google 문서상 금지되어 있다. Gemini 통합은 기본적으로 Vertex AI 또는 Google AI Studio API key/서비스 계정 경로를 우선해야 한다. [R12]

최종 설계 명칭

이 기능은 “OAuth Provider”가 아니라 “Local CLI Provider / Bring-your-own-authenticated-CLI / User-owned CLI session”으로 명명하는 것이 정확하다.

2. 범위와 전제

2.1 지원하려는 이용 방식

구분	방식	Aichestra에서의 위치	핵심 포인트
API Key	OpenAI/Anthropic/Gemini API key	cloud_api Provider	정적 secret 저장·회전·비용 추적 필요
OAuth 사용자 로그인	Codex ChatGPT OAuth/device, Gemini OAuth 등	oauth_user Provider 또는 CLI 외부 세션	사용자/조직 라이선스와 권한을 따른다
WIF/OIDC	Claude WIF, cloud workload identity	workload_identity Provider	정적 API key 없이 short-lived token 사용
Cloud IAM/ADC	Vertex AI ADC, Bedrock IAM, Foundry 등	cloud_iam Provider	클라우드 프로젝트·서비스 계정·IAM 정책과 결합
Local CLI	claude, codex, gemini, 기타 기업형 CLI	local_cli Provider	Aichestra Local Agent가 프로세스 실행과 IO 중계
PTY fallback	interactive TUI 자동화	pty_interactive Provider	불안정하므로 최후의 수단

2.2 비목표

- Provider의 약관·라이선스·조직 정책을 우회하는 인증 대체 수단을 만들지 않는다.
- 사용자의 로컬 CLI 인증 토큰을 수집, 복사, 재사용, 공유하지 않는다.
- 한 사용자의 CLI 세션을 여러 사용자에게 중계하는 shared credential gateway를 만들지 않는다.
- interactive terminal 화면을 기본 통합 수단으로 삼지 않는다.

2.3 기본 가정

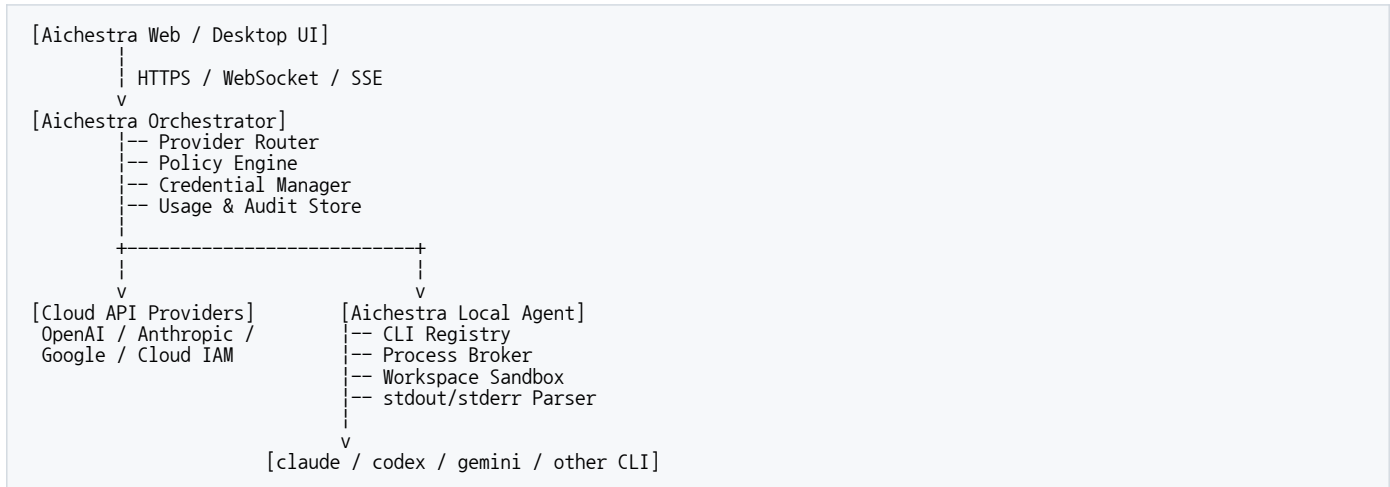
- 사용자 컴퓨터에는 터미널과 필요한 CLI가 이미 설치되어 있을 수 있다.

- Aichestra Local Agent 는 사용자 승인 하에 특정 workspace 디렉터리 안에서만 CLI 를 실행한다.
- CLI 호출 결과는 stdout/stderr/exit code 또는 JSON/JSONL 이벤트로 수집한다.
- 프로세스 실행 권한, 파일 접근 권한, 네트워크 접근 권한은 Provider 별 정책 엔진이 제한한다.

3. 통합 아키텍처

3.1 전체 구조

그림 1. Aichestra Provider 통합 구조



핵심은 Provider 호출 계층과 인증 계층을 분리하는 것이다. Provider Router 는 어떤 Provider 를 쓸지 결정하고, Credential Manager 는 API key, OAuth token, WIF token, cloud IAM credential, external CLI session 을 처리한다. Local CLI Provider 는 Aichestra Local Agent 를 통해서만 접근한다.

3.2 Plane 분리

Plane	역할	대표 컴포넌트	주의사항
Provider Plane	모델 호출·응답 변환	Provider Router, Adapter	Provider 별 출력 포맷 차이를 normalize
Credential Plane	인증 획득·갱신·보관	Credential Manager, Token Resolver	CLI auth cache 는 직접 읽지 않음
Execution Plane	로컬 프로세스 실행·격리	Local Agent, Process Broker, Sandbox	shell=false, cwd allowlist, timeout 필수
Policy Plane	권한·승인·위험 제어	Policy Engine, Approval Gate	파일 쓰기·명령 실행·네트워크 접근 제한
Observability Plane	비용·사용량·감사 추적	Audit Log, Usage Meter	Provider 별 과금 주제를 명확히 기록

3.3 Provider Router 원칙

- Aichestra 내부 Task 는 “모델 호출”을 요청하고, 실제 인증/실행 방식은 Provider Adapter 가 처리한다.
- Provider 설정은 authType 이 아니라 capability 와 policy 를 함께 가져야 한다.
- Local CLI Provider 는 cloud provider 와 동일한 인터페이스로 보이되, 내부적으로 process lifecycle 을 관리한다.
- CLI 가 JSON/JSONL 을 제공하면 structured parser 를 우선하고, plain text stdout 은 마지막 단계에서만 사용한다.

4. Provider 접근 방식 총괄

4.1 접근 방식 비교

접근 방식	대표 예시	장점	단점/제약	권장도
API key	OpenAI API, Claude Console, Gemini API key	구현 단순, 자동화에 적합	장기 secret 보관·회전 필요	높음
OAuth user/device	Codex login, Google OAuth	사용자 계정·조직 라이선스 활용	자동화/제 3 자 중계 제약 가능	중간
WIF/OIDC	Claude WIF	정적 API key 제거, short-lived token	IdP·rule·service account 설정 필요	기업 환경에서 높음
Cloud IAM/ADC	Vertex AI ADC, Bedrock IAM	클라우드 거버넌스·billing 통합	클라우드 프로젝트 설정 필요	기업 운영에서 높음
Local CLI headless	claude -p, codex exec, gemini -p	사용자의 설치·로그인 상태 활용, UI 연동 쉬움	CLI 버전·정책·출력 포맷 변경 리스크	조건부 높음
PTY interactive	TUI 화면 자동화	headless 미지원 CLI 도 연결 가능	ANSI/TUI 파싱 불안정, 승인 프롬프트 hang 위험	낮음

4.2 권장 우선순위

1. 공식 API/SDK 가 있으면 우선 사용한다. 비용·rate limit·감사·권한 관리가 가장 예측 가능하다.
2. 정적 secret 을 줄여야 하는 기업 환경에서는 OAuth/WIF/IAM 을 적용한다.
3. 사용자 PC 의 인증된 CLI 를 그대로 활용해야 하는 경우 Local CLI Provider 를 사용한다.
4. CLI headless mode 가 없거나 구조화 출력이 없을 때만 PTY 기반 interactive automation 을 검토한다.

4.3 과금과 권한의 해석

방식	과금/한도 귀속	Aichestra 가 해야 할 일
API key	API key 가 속한 platform account/workspace	key owner, quota, spend limit, project id 기록
OAuth user	로그인한 사용자 또는 조직 플랜/라이선스	사용자 동의, org policy, rate limit 표시
WIF/IAM	service account, workspace, cloud project	federation rule, service account, project/location 기록
Local CLI	해당 CLI 가 선택한 인증 방식	CLI 인증을 추측하지 말고 external session 으로 기록

과금 회피 금지

OAuth 또는 Local CLI 방식은 API 과금을 회피하기 위한 설계가 아니다. underlying Provider 의 계정, 플랜, workspace, 프로젝트, rate limit, 약관을 그대로 따른다.

5. Provider 별 설계

5.1 OpenAI Codex / Codex CLI

Codex 는 ChatGPT OAuth/device auth 또는 API key 로 로그인할 수 있으며, CLI 자동화에는 codex exec 가 적합하다. Codex 문서에 따르면 codex exec 는 진행 상황을 stderr 로 스트리밍하고 최종 agent message 를 stdout 으로 출력하며, --json 사용 시 stdout 은 JSONL 이벤트 스트림이 된다. [R1][R2][R3]

항목	설계 결정
지원 모드	cloud_api, oauth_user, local_cli, codex_sdk
권장 호출	codex exec --json "<prompt>" 또는 cat prompt.txt codex exec - --json
인증	codex login 으로 저장된 CLI 세션을 재사용하되 Aichestra 는 auth.json 을 읽지 않음
자동화 주의	OpenAI 문서상 프로그램형 Codex CLI workflow 는 API key 인증이 권장됨. ChatGPT-managed auth 자동화는 trusted private runner 에서만 고급 워크 플로로 취급 [R2][R4]
보안	danger-full-access 는 차단하거나 별도 관리자 정책으로만 허용

Codex CLI 호출 예시

```
# final message 만 stdout 으로 받고 싶은 기본 패턴
codex exec "summarize the repository structure"

# JSONL 이벤트 스트림으로 수집
codex exec --json "summarize the repo and list risks" > result.jsonl

# stdin 을 프롬프트로 사용
cat prompt.txt | codex exec - --json > result.jsonl
```

5.2 Anthropic Claude / Claude Code

Claude Code 는 claude -p 를 통해 비대화형 실행을 지원하고, text/json/stream-json 출력 포맷을 제공한다. stdin pipe 도 지원하며, stream-json 은 실시간 이벤트 수집에 적합하다. [R5]

항목	설계 결정
지원 모드	api_key, oauth_user, workload_identity, cloud_iam, local_cli
권장 호출	claude -p "<prompt>" --output-format json 또는 stream-json
인증	Claude.ai/Teams/Enterprise/Console/cloud provider 인증을 CLI 가 처리. Aichestra 는 외부 세션으로만 취급 [R6]
WIF	Anthropic WIF 는 IdP JWT 를 POST /v1/oauth/token 에 교환해 short-lived access token 을 사용하는 구조 [R7][R8]
주의	--bare 는 스크립트/SDK 호출에 유리하지만 OAuth/keychain read 를 건너뛰므로 API key 또는 apiKeyHelper 등 별도 인증 설정이 필요할 수 있음 [R5]

Claude Code 호출 예시

```
# plain text
claude -p "What does the auth module do?"

# JSON 응답
claude -p "Summarize this project" --output-format json
```

```
# 실시간 JSONL 스트리밍
claude -p "Explain recursion" \
  --output-format stream-json \
  --verbose \
  --include-partial-messages

# stdin pipe
cat build-error.txt | claude -p "explain the root cause" > output.txt
```

5.3 Google Gemini / Gemini CLI / Vertex AI

Gemini Developer API 는 API key 가 가장 쉬운 인증 방식이고 stricter access control 이 필요하면 OAuth 를 쓸 수 있다. Vertex AI 에서 Gemini 를 사용할 때는 테스트에는 API key, 운영에는 Application Default Credentials(ADC)가 권장된다. [R9][R13]

항목	설계 결정
지원 모드	api_key, oauth_user, cloud_iam/ADC, local_cli
권장 enterprise 경로	Vertex AI + ADC/service account 또는 Google AI Studio/Vertex API key
CLI 인증	Gemini CLI 는 Login with Google, Gemini API key, Vertex AI 인증을 지원 [R10]
CLI non-interactive	gemini -p "<prompt>" --output-format json [R11]
정책 주의	Gemini CLI OAuth 세션을 제 3 자 소프트웨어가 harvest/piggyback 하여 backend service 에 접근하는 것은 Google 문서상 금지. Aichestra 는 Gemini OAuth-CLI 자동화를 기본 비활성화하고, API key/Vertex/엔터프라이즈 승인 경로를 기본값으로 둔다. [R12]

Gemini CLI 호출 예시

```
# Gemini CLI non-interactive structured output
# 기본 권장: GEMINI_API_KEY 또는 Vertex AI/ADC 환경이 명확히 설정된 경우
export GEMINI_API_KEY="..."
gemini -p "Summarize this repository" --output-format json

# Vertex AI 환경 변수 예시
export GOOGLE_GENAI_USE_VERTEXAI=true
export GOOGLE_CLOUD_PROJECT="my-gcp-project"
export GOOGLE_CLOUD_LOCATION="us-central1"
gemini -p "Review this diff" --output-format json
```

Gemini CLI 정책 기본값

Aichestra 정책 기본값은 gemini CLI 의 OAuth-login 세션을 자동 통합 대상으로 보지 않는 것이다. 사용자 로컬 CLI 를 실행하더라도, Google 문서가 금지하는 OAuth piggyback/harvest 패턴이 되지 않도록 enterprise approval 또는 API key/Vertex 경로를 요구한다.

5.4 기타 CLI 형 기업형 LLM

기타 CLI 는 다음 조건을 만족하면 Local CLI Provider 로 흡수할 수 있다.

- non-interactive/headless mode 가 존재한다.
- stdin 또는 파일 입력을 받을 수 있다.
- stdout/stderr/exit code 또는 JSON/JSONL 출력을 제공한다.
- 인증 저장소를 직접 노출하지 않고 CLI 자체가 인증을 처리한다.
- 사용자 승인과 workspace sandbox 안에서 실행할 수 있다.

6. Local CLI Provider 상세 설계

6.1 컴포넌트

컴포넌트	책임	실패 시 동작
CLI Registry	설치 여부, path, version, capabilities 감지	Provider unavailable 상태 표시
Process Broker	spawn, stdin write, stdout/stderr stream, timeout 관리	process kill, retry policy 적용
Parser	raw/json/jsonl/ndjson 이벤트 normalize	parse 실패 시 raw capture 로 fallback
Policy Gate	실행 전 권한·cwd·파일 쓰기·shell 명령 승인	승인 거부 또는 human approval 요청
Workspace Sandbox	cwd allowlist, file access, network/shell 제한	위반 시 execution block
Audit Logger	prompt metadata, command args, exit code, usage, 비용 hints 기록	민감 정보 redaction 후 저장

6.2 실행 수명주기

1. Provider Router 가 local_cli Provider 를 선택한다.
2. Policy Engine 이 workspace, tool permission, file write, shell execution 권한을 평가한다.
3. CLI Registry 가 command path 와 version 을 확인한다.
4. Prompt Assembler 가 task prompt, context, system hint 를 생성한다.
5. Process Broker 가 shell=false 로 CLI 를 spawn 한다.
6. stdinMode 에 따라 context/prompt 를 stdin 으로 주입한다.
7. stdout/stderr 를 스트리밍 수집하고 Parser 가 Aichestra event 로 normalize 한다.
8. exit code 와 final output 을 ProviderResult 로 반환한다.
9. Audit Logger 가 민감 정보 제거 후 execution summary 를 저장한다.

6.3 stdout/stderr 처리 원칙

채널	의미	처리 방식
stdout	최종 답변, JSON, JSONL 이벤트	ProviderResult.content 또는 event stream 으로 사용
stderr	진행 로그, 경고, 일부 CLI 의 progress	UI progress log 로 표시하되 final answer 와 분리
exit code	성공/실패 판단	0 이면 success, non-zero 는 ProviderError 생성
timeout	무한 대기 방지	SIGTERM 후 grace period, 필요 시 SIGKILL

6.4 Headless 우선, PTY 는 fallback

headless mode 는 출력 구조가 안정적이고 자동화에 적합하다. PTY/TUI 자동화는 ANSI escape, alternate screen, spinner, 승인 프롬프트, UI 변경에 취약하므로 fallback 으로만 둔다.

모드	사용 조건	장점	제약
headless	CLI 가 -p/exec/non-interactive 지원	안정적, parser 가능, CI 친화적	CLI 별 플래그 관리 필요
streaming	JSONL/NDJSON stream 제공	Aichestra UI 에 실시간 출력 가능	parser 와 backpressure 필요

모드	사용 조건	장점	제약
pty_interactive	headless 미지원 또는 수동 승인 필요	대부분 CLI 화면 제어 가능	불안정, 테스트 비용 높음

6.5 Local Agent 가 필요한 이유

브라우저 기반 Aichestra Web UI 는 사용자 컴퓨터의 터미널 프로세스를 직접 실행할 수 없다. 따라서 Cloud/Web UI 가 Local CLI Provider 를 쓰려면 사용자 PC 에서 실행되는 Aichestra Local Agent 가 필요하다. Desktop 앱이라면 Local Agent 역할을 앱 내부에 포함할 수 있다.

Local Agent 데이터 흐름

```

Aichestra Cloud/Web UI
  |
  | signed task request
  v
Aichestra Local Agent on user machine
  |-- verifies user consent and policy
  |-- spawns CLI process in allowed workspace
  |-- streams normalized events back to UI
  v
local enterprise LLM CLI

```

7. Credential, 보안, 정책 모델

7.1 Credential 추상화

ProviderAuth 타입

```

type ProviderAuth =
  { type: "api_key"; secretRef: string }
  { type: "oauth_user"; provider: string; scopes?: string[] }
  { type: "device_code"; provider: string; scopes?: string[] }
  { type: "workload_identity"; provider: string; ruleId?: string; serviceAccountId?: string }
  { type: "cloud_iam"; provider: "vertex_ai" | "bedrock" | "foundry"; project?: string; location?: string }
  { type: "external_cli_session"; credentialAccess: "never_read_tokens" };

```

7.2 절대 금지 패턴

- CLI auth cache 파일을 읽어서 API 호출에 재사용한다.
- 사용자 로컬 credential 을 Aichestra 서버에 업로드한다.
- 한 사용자의 OAuth/CLI 세션을 다른 사용자 작업에 공유한다.
- Provider 약관상 금지된 OAuth piggyback/harvest 를 구현한다.
- danger-full-access, yolo, auto-approve-all 을 기본값으로 둔다.
- shell=true 로 사용자 입력을 command string 에 직접 결합한다.
- 승인 없이 workspace 밖 파일을 읽거나 쓴다.

7.3 권한 정책

정책 항목	기본값	승격 조건
파일 읽기	workspace 내 read 허용	workspace 밖은 사용자 명시 승인
파일 쓰기	기본 차단 또는 diff preview 필요	사용자/관리자 승인 후 workspace-write
shell 명령	기본 차단	allowlist 또는 task-specific approval
네트워크 접근	CLI/Provider 기본 동작만 허용	외부 endpoint 호출은 별도 정책
비용 발생	호출 전 Provider 와 과금 귀속 표시	관리자 budget 또는 사용자 승인

정책 항목	기본값	승격 조건
민감정보	prompt/context redaction 적용	필요 시 ephemeral mode 또는 no-store mode

7.4 Local Agent 보안 경계

- **상호 인증:** Cloud/Web UI 와 Local Agent 사이에는 device pairing, signed task, short-lived session key 를 사용한다.
- **명령 allowlist:** Provider config 의 command 와 argsTemplate 만 실행한다. 사용자 입력은 별도 argv 로 전달하고 shell interpolation 을 금지한다.
- **환경 변수 allowlist:** 기본적으로 HOME, PATH, LANG 등 최소 변수만 전달하고, Provider 별 필요한 변수만 명시 허용한다.
- **로그 redaction:** API key, bearer token, auth file path, 이메일, 프로젝트 secret 을 로그에서 마스킹한다.
- **사용자 승인:** 첫 실행, 파일 쓰기, shell 명령, workspace 밖 접근, long-running execution 은 승인 UX 를 요구한다.

7.5 감사 로그 모델

감사 로그 타입

```
interface ProviderExecutionAudit {
  executionId: string;
  providerId: string;
  providerKind: "cloud_api" | "oauth_api" | "local_cli";
  authType: ProviderAuth["type"];
  userId: string;
  workspaceId: string;
  command?: string;      // local_cli only; args are redacted
  cwd?: string;           // normalized, allowlisted path
  startedAt: string;
  endedAt?: string;
  exitCode?: number;
  policyDecision: "allow" | "deny" | "needs_approval";
  usage?: { inputTokens?: number; outputTokens?: number; costHint?: string };
  redactionApplied: boolean;
}
```

8. 설정 스키마와 예시

8.1 공통 Provider 설정

ProviderConfig 타입

```
interface ProviderConfig {
  id: string;
  displayName: string;
  kind: "cloud_api" | "oauth_api" | "local_cli";
  vendor: "openai" | "anthropic" | "google" | "custom";
  model?: string;
  auth: ProviderAuth;
  capabilities: {
    streaming: boolean;
    structuredOutput: boolean;
    toolUse: boolean;
    fileRead: boolean;
    fileWrite: boolean;
    shellExecution: boolean;
  };
  policy: ProviderPolicy;
}
```

8.2 Local CLI Provider 설정

LocalCliProviderConfig 타입

```
interface LocalCliProviderConfig extends ProviderConfig {
  kind: "local_cli";
}
```

```

command: "claude" | "codex" | "gemini" | string;
commandDiscovery?: {
  strategy: "path" | "which" | "configured_path";
  configuredPath?: string;
  versionArgs?: string[];
  minVersion?: string;
};
invocation: {
  argsTemplate: string[];
  stdinMode: "none" | "prompt" | "context" | "prompt_plus_context";
  stdoutParser: "raw" | "json" | "jsonl" | "ndjson";
  stderrPolicy: "progress_log" | "error_only" | "discard";
  timeoutMs: number;
};
cwdPolicy: {
  type: "workspace_only" | "user_selected_dir";
  allowedDirs: string[];
};
}

```

8.3 YAML 예시

Provider YAML 설정 예시

```

providers:
- id: claude-code-local
  displayName: Claude Code Local
  kind: local_cli
  vendor: anthropic
  command: claude
  auth:
    type: external_cli_session
    credentialAccess: never_read_tokens
  invocation:
    argsTemplate:
      - "-p"
      - "{{prompt}}"
      - "--output-format"
      - "stream-json"
      - "--verbose"
      - "--include-partial-messages"
    stdinMode: context
    stdoutParser: ndjson
    stderrPolicy: progress_log
    timeoutMs: 300000
  cwdPolicy:
    type: workspace_only
    allowedDirs: ["{{workspaceRoot}}"]
  capabilities:
    streaming: true
    structuredOutput: true
    toolUse: true
    fileRead: true
    fileWrite: false
    shellExecution: false

- id: codex-local
  displayName: Codex CLI Local
  kind: local_cli
  vendor: openai
  command: codex
  auth:
    type: external_cli_session
    credentialAccess: never_read_tokens
  invocation:
    argsTemplate: ["exec", "--json", "{{prompt}}"]
    stdinMode: context
    stdoutParser: jsonl
    stderrPolicy: progress_log
    timeoutMs: 300000
  cwdPolicy:
    type: workspace_only
    allowedDirs: ["{{workspaceRoot}}"]
  capabilities:
    streaming: true
    structuredOutput: true
    toolUse: true
    fileRead: true
    fileWrite: false
    shellExecution: false

```

```

- id: gemini-vertex-cli
  displayName: Gemini CLI via Vertex AI
  kind: local_cli
  vendor: google
  command: gemini
  auth:
    type: external_cli_session
    credentialAccess: never_read_tokens
  requiredEnv:
    - GOOGLE_GENAI_USE_VERTEXAI
    - GOOGLE_CLOUD_PROJECT
    - GOOGLE_CLOUD_LOCATION
  invocation:
    argsTemplate: ["-p", "{{prompt}}", "--output-format", "json"]
    stdinMode: context
    stdoutParser: json
    stderrPolicy: progress_log
    timeoutMs: 300000
  policy:
    requireEnterpriseApprovalForOAuthSession: true

```

9. 구현 플로우와 코드 예시

9.1 Node.js Process Broker 예시

Process Broker 최소 구현

```

import { spawn } from "node:child_process";

export async function runCliProvider(input: {
  command: string;
  args: string[];
  stdin?: string;
  cwd: string;
  env: Record<string, string>;
  timeoutMs: number;
}): Promise<{ stdout: string; stderr: string; exitCode: number | null }> {
  return await new Promise((resolve, reject) => {
    const child = spawn(input.command, input.args, {
      cwd: input.cwd,
      shell: false,
      env: input.env,
    });

    let stdout = "";
    let stderr = "";

    const timer = setTimeout(() => {
      child.kill("SIGTERM");
      reject(new Error("CLI provider timed out"));
    }, input.timeoutMs);

    child.stdout.on("data", chunk => {
      stdout += chunk.toString("utf8");
      // stream chunk to Aichestra UI after parser normalization
    });

    child.stderr.on("data", chunk => {
      stderr += chunk.toString("utf8");
      // progress log, not final answer
    });

    child.on("error", err => {
      clearTimeout(timer);
      reject(err);
    });

    child.on("close", exitCode => {
      clearTimeout(timer);
      resolve({ stdout, stderr, exitCode });
    });

    if (input.stdin) child.stdin.write(input.stdin);
    child.stdin.end();
  });
}

```

9.2 Parser 설계

stdoutParser	사용 Provider	처리
raw	Codex final stdout, 단순 CLI	stdout 전체를 final text 로 반환
json	Claude --output-format json, Gemini --output-format json	JSON.parse 후 result/text/structured_output 추출
jsonl/ndjson	Codex --json, Claude stream-json	line 단위 parse 후 event bus 로 전달

9.3 오류 처리

오류 유형	탐지	대응
CLI not found	spawn error ENOENT	설치 가이드와 command path 설정 UX 제공
인증 실패	stderr/stdout pattern, non-zero exit	사용자에게 CLI login 또는 env 설정 안내
권한 프롬프트 hang	timeout/no output watchdog	프로세스 종료 후 approval UX 표시
JSON parse 실패	parser exception	raw capture 저장, Provider adapter 버전 mismatch 표시
Rate limit	Provider-specific error	retry-after/backoff, 사용자 quota 안내
정책 위반	Policy Engine deny	실행하지 않고 사유 표시

9.4 ProviderResult 표준화

ProviderResult 타입

```
interface ProviderResult {
  providerId: string;
  status: "success" | "failed" | "cancelled";
  content?: string;
  structured?: unknown;
  events?: ProviderEvent[];
  usage?: {
    inputTokens?: number;
    outputTokens?: number;
    costUsd?: number;
    raw?: unknown;
  };
  diagnostics?: {
    stderr?: string;
    exitCode?: number | null;
    commandVersion?: string;
  };
}
```

10. 테스트, 운영, 단계별 로드맵

10.1 테스트 전략

테스트 유형	목표	예시
Unit	argsTemplate, env allowlist, parser 검증	JSONL fixture parse, shell injection 차단
Integration	실제 CLI smoke test	claude -p, codex exec --json, gemini -p 실행
Security	권한·경로·secret redaction 검증	../ path, auth file read 시도 차단

테스트 유형	목표	예시
Policy	승인 필요 작업 검증	file write, shell execution, danger mode 차단
Regression	CLI 버전별 출력 포맷 변경 감지	golden stdout/stderr fixture 비교
User UX	로그인 실패/미설치/권한 승인 흐름 검증	설치 가이드, login 안내, timeout 안내

10.2 운영 모니터링

- Provider 별 success/failure rate, 평균 latency, timeout rate 를 추적한다.
- local_cli 는 command version, parser version, policy decision 을 함께 기록한다.
- 비용은 API key/WIF/IAM 경로에서 가능한 한 token/cost 를 수집하고, CLI 외부 세션은 costHint 또는 provider-reported usage 만 기록한다.
- 민감정보 redaction 이 적용되지 않은 로그는 저장하지 않는다.

10.3 단계별 구현 로드맵

Phase	범위	산출물	완료 기준
0	Provider taxonomy 정리	ProviderConfig/Auth schema	API key/OAuth/local_cli 를 config 로 표현
1	Cloud API + OAuth/WIF/IAM	Credential Manager, Token Resolver	Claude WIF, Vertex ADC, API key smoke test
2	Local CLI Provider MVP	Local Agent, Process Broker, Parser	Claude/Codex headless 성공, Gemini API-key/Vertex 경로 성공
3	Policy/Approval UX	파일 쓰기·shell 실행 승인 flow	기본 read-only, workspace-write 승인 가능
4	PTY fallback	node-pty adapter, TUI parser fixture	headless 미지원 CLI 만 제한 적용
5	Enterprise governance	audit, budget, admin policy, org config	관리자 정책으로 Provider 별 허용/차단 가능

11. 리스크와 의사결정 표

11.1 주요 리스크

리스크	영향	완화책
CLI 출력 포맷 변경	Parser 실패, UI 깨짐	버전 감지, golden fixture, raw fallback
OAuth 정책 위반	계정 정지/법무 리스크	Provider 별 policy registry, Gemini OAuth-CLI 기본 차단
토큰 유출	계정 탈취	never_read_tokens, env allowlist, log redaction
무단 파일 수정	데이터 손실	workspace sandbox, diff preview, approval gate
명령 실행 악용	로컬 시스템 손상	shell=false, command allowlist, deny danger/yolo default

리스크	영향	완화책
비용 폭증	예산 초과	quota, budget, rate limit, pre-flight estimate
순수 웹에서 로컬 접근 불가	기능 미작동	Local Agent pairing 필수

11.2 의사결정 요약

결정	채택안	근거
Provider 분류	cloud_api / oauth_api / local_cli	인증과 실행 경로가 다르므로 단일 API Provider 로 묶으면 위험
CLI 통합 방식	headless 우선	stdout/stderr/JSON 구조가 안정적이고 공식 자동화 모드와 일치
Credential 접근	external_cli_session + never_read_tokens	토큰 수집 없이 사용자 CLI 세션을 사용
Gemini CLI OAuth	기본 비활성화/승인 필요	Google 문서의 OAuth piggyback 금지와 충돌 방지
PTY 자동화	fallback only	TUI 파싱 불안정성과 승인 프롬프트 hang 위험
과금	underlying Provider 귀속	OAuth/CLI 는 과금 회피 수단이 아님

11.3 권장 기본 정책

Default Policy

Aichestra Local CLI Provider 는 read-only, workspace-only, shell-execution=false, file-write=false, never-read-tokens, structured-output-preferred 를 기본값으로 한다. 파일 쓰기·shell 실행·danger/yolo 류 권한은 사용자 및 관리자 승인 없이는 허용하지 않는다.

12. 참고 출처

아래 출처는 2026-05-11 기준 공식 문서 또는 Provider 공식 GitHub 문서를 기준으로 확인했다. 약관·정책·CLI 플래그는 수시로 변경될 수 있으므로 실제 릴리스 전 재검증이 필요하다.

ID	문서	URL	사용 근거
R1	OpenAI Developers - Codex Authentication	https://developers.openai.com/codex/auth	Codex ChatGPT/API key 로그인, API key 과금, 프로그램형 workflow 권장 사항
R2	OpenAI Developers - Codex Non-interactive mode	https://developers.openai.com/codex/noninteractive	codex exec, stdout/stderr, JSONL, sandbox, stdin piping, CI 인증
R3	OpenAI Developers - Codex CLI Reference	https://developers.openai.com/codex/cli/reference	codex login OAuth/device/API key, CLI commands

ID	문서	URL	사용 근거
R4	OpenAI Developers - Maintain Codex account auth in CI/CD	https://developers.openai.com/codex/auth/ci-cd-auth	ChatGPT-managed Codex auth 를 trusted CI 에서 유지하는 고급 패턴
R5	Claude Code Docs - Run Claude Code programmatically	https://code.claude.com/docs/en/headless	claude -p, stdin, json/stream-json, bare mode, streaming
R6	Claude Code Docs - Authentication	https://code.claude.com/docs/en/iam	Claude.ai, Teams/Enterprise, Console, Bedrock, Vertex AI, Foundry 인증
R7	Claude API Docs - Authentication	https://platform.claude.com/docs/en/manage-claude/authentication	API key 와 Workload Identity Federation 인증 개요
R8	Claude API Docs - Workload Identity Federation	https://platform.claude.com/docs/en/manage-claude/workload-identity-federation	IdP JWT 를 POST /v1/oauth/token 으로 교환하는 구조
R9	Google AI for Developers - Gemini API OAuth quickstart	https://ai.google.dev/gemini-api/docs/oauth	Gemini API OAuth 인증과 API key 대비 사용 조건
R10	Gemini CLI Docs - Authentication Setup	https://google-gemini.github.io/gemini-cli/docs/get-started/authentication.html	Login with Google, Gemini API key, Vertex AI, headless 인증
R11	Gemini CLI Docs - Configuration	https://google-gemini.github.io/gemini-cli/docs/get-started/configuration.html	gemini -p, --output-format json, env vars, approval modes
R12	google-gemini/gemini-cli FAQ	https://github.com/google-gemini/gemini-cli/blob/main/docs/resources/faq.md	Gemini CLI OAuth authentication piggyback 금지와 API key/Vertex 권장
R13	Google Cloud Docs - Configure ADC for Vertex AI Gemini	https://docs.cloud.google.com/vertex-ai/generative-ai/docs/start/gcp-auth	Vertex AI Gemini 인증: 테스트 API key, 운영 ADC 권장

ID	문서	URL	사용 근거
R14	OpenAI Developers - Codex SDK	https:// developers.openai.com/ codex/sdk	Codex local agents 를 SDK 로 제어하는 대안

부록 A. 빠른 적용 체크리스트

- ☐ ProviderConfig 에 kind=local_cli 와 auth.type=external_cli_session 을 추가했는가?
- ☐ Local Agent 가 사용자 PC 에서 실행되고, Cloud/Web UI 와 안전하게 pairing 되는가?
- ☐ CLI command/args 가 allowlist 기반이고 shell=false 로 실행되는가?
- ☐ stdout/stderr/exit code 를 분리하고 parser fallback 을 제공하는가?
- ☐ CLI auth cache 를 읽지 않고 로그에 토큰이 남지 않는가?
- ☐ Gemini OAuth-CLI 자동화는 기본 차단 또는 enterprise approval 로 제한되는가?
- ☐ 파일 쓰기와 shell 실행은 사용자 승인과 관리자 정책을 통과해야 하는가?
- ☐ Provider 별 비용·rate limit·과금 귀속이 UI 와 audit log 에 표시되는가?